CSCI 112 Assignment – Standard Library Functions II

Points: 10

Automated tests will run on GitHub Actions after every push. Check the Actions tab to see which tests pass and which fail.

Instructions

1. **Fork** the repository on GitHub - `https://github.com/jpach-cs/112-ass04-template`.
2. Clone your forked repository to your local machine.
3. Open the file `code.c`.
4. Implement every function listed below:
   - Read the comments above each function carefully.
   - Follow the rules — use ONLY `if`, `switch`, `goto`, `putchar()`, `getchar()`, `sprintf()`.
   - Do NOT use `while`, `for`, `do-while`, or any standard library functions.
   - `my_isalpha()` MUST call your own `my_islower()` and `my_isupper()`.
   - `my_isalnum()` MUST use `switch()` and call your own `my_isalpha()` and `my_isdigit()`.
   - Every format function MUST call `clear_string(dest, 64)` as its first statement.
5. Test your implementation locally using `main.c`.
6. Once all functions are complete — save your changes, then **commit** and **push** to your repository.
7. Go to the **Actions** tab on GitHub and check the test results.

Functions to implement

Utility

Function	Description
`void clear_string(char s[], int n)`	Sets first `n` characters of `s` to `'\0'`. MUST use `goto`.

From previous lab — re-implement

Function	Description
`int my_isdigit(char c)`	Returns `1` if `c` is `'0'`–`'9'`, otherwise `0`.
`int my_islower(char c)`	Returns `1` if `c` is `'a'`–`'z'`, otherwise `0`.

New functions

Function	Description
`int my_isupper(char c)`	Returns `1` if `c` is `'A'`–`'Z'`, otherwise `0`.
`int my_isalpha(char c)`	Returns `1` if `c` is a letter. MUST call `my_islower()` and `my_isupper()`.
`int my_isalnum(char c)`	Returns `1` if `c` is a letter or digit. MUST use `switch()`.
`int my_strcmp(char a[], char b[])`	Returns `-1`, `0`, or `1`.
`int my_strchr(char s[], char c)`	Returns index of first occurrence of `c`, or `-1`.
`int my_pow(int a, int b)`	Returns `a` raised to the power of `b`.
`double my_pow_double(double a, int b)`	Returns `a` raised to the power of `b`.

Format functions

Function	Description
`char * format_my_isupper(char dest[], char c, int r)`	e.g. `"isupper('A') = true"`
`char * format_my_isalpha(char dest[], char c, int r)`	e.g. `"isalpha('a') = true"`
`char * format_my_isalnum(char dest[], char c, int r)`	e.g. `"isalnum('!') = false"`
`char * format_my_strcmp(char dest[], int r)`	e.g. `"comparison: equal"`
`char * format_my_strchr(char dest[], int r)`	e.g. `"found at: 2"` or `"not found"`
`char * format_my_pow(char dest[], int r)`	e.g. `"pow = 256 "` — left-justified, width 12
`char * format_my_pow_double(char dest[], double r)`	fixed total width of 12, adaptive precision

Iteration pattern — reminder

```
int i = 0;
loop:
    if (i >= n)
        goto done;
    // body
    i++;
    goto loop;
done:
```

#Header comment — required in code.c

```
// Your Name
// CSCI 112 Fall 2026
// Programming Assignment #1
// I declare that I am the author of this work, take full responsibility for it, and have d
```